PHẦN 2: CÔNG NGHỆ CỐT LÕI (CNN), THỰC TIỄN & CÔNG CỤ

Từ Kiến trúc Xử lý Ảnh đến Hệ sinh thái Lập trình

Trong phần này, chúng ta sẽ đi sâu vào:

1. **Công nghệ cốt lõi:** Khám phá Mạng Nơ-ron Tích chập (CNN) - "vũ khí" chuyên dụng cho các bài toán thị giác máy tính.
2. **Bằng chứng thực tiễn:** Phân tích các nghiên cứu điển hình về việc áp dụng CNN để phát hiện bệnh trên lá khoai tây và táo.
3. **Công cụ trong tay bạn:** Làm quen với hệ sinh thái Keras & TensorFlow để tự xây dựng, huấn luyện và tối ưu hóa các mô hình học sâu.

NotebookLM

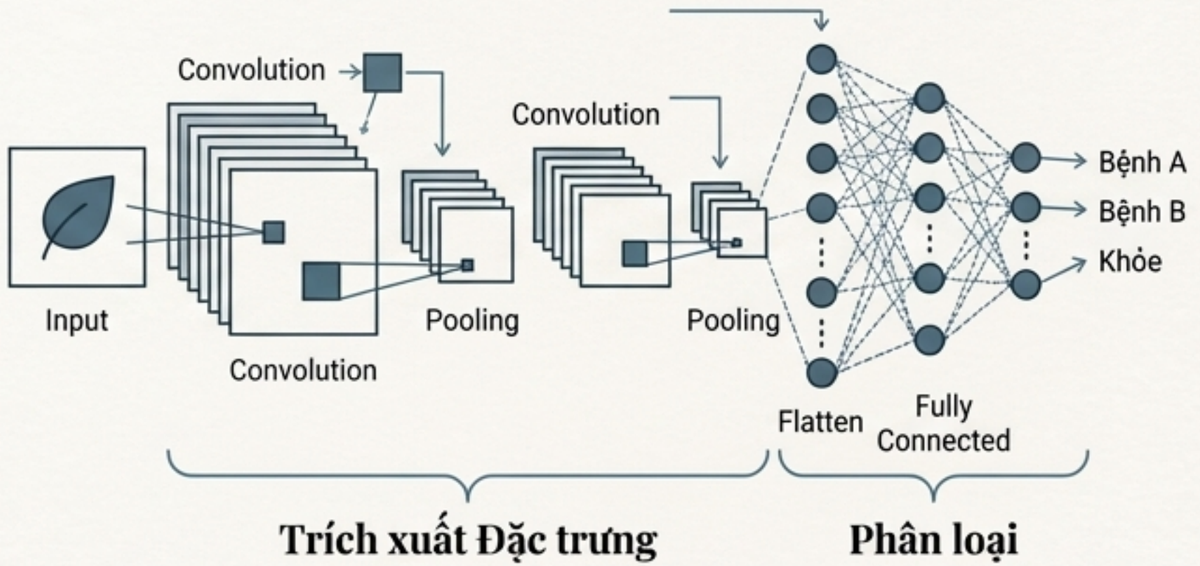
Mạng Nơ-ron Tích chập (CNN): "Vũ khí" cho Thị giác Máy tính

Thách thức của ANN thông thường với hình ảnh:

- **Mất thông tin không gian:** Việc 'làm phẳng' (flatten) ảnh thành vector 1D phá vỡ mối quan hệ giữa các pixel lân cận, làm mất đi các đặc trưng như cạnh, góc, hoa văn.
- **Bùng nổ tham số:** Khi kích thước ảnh tăng, số lượng tham số có thể huấn luyện tăng lên đáng kể, dẫn đến mô hình quá lớn, dễ bị quá khớp (overfitting) và tốn kém tài nguyên.

Giải pháp của CNN:

- **Lớp Tích chập (Convolutional Layer):** Sử dụng các 'bộ lọc' (filter/kernel) để quét qua ảnh, tự động học và trích xuất các đặc trưng không gian.
- **Lớp Gộp (Pooling Layer):** Giảm kích thước của bản đồ đặc trưng một cách thông minh, giữ lại những thông tin quan trọng nhất, giúp mô hình khái quát hóa tốt hơn và giảm chi phí tính toán.



NotebookLM

Nghiên cứi Thực tiễn

CNN đã chứng minh hiệu quả trong việc phát hiện bệnh cây trồng như thế nào?

Chúng ta sẽ phân tích hai trường hợp ứng dụng tiêu biểu để thấy rõ sức mạnh của CNN:
1. **Phân loại bệnh trên lá Khoai tây:** Xây dựng một mô hình CNN từ đầu để giải quyết một bài toán cụ thể với độ chính xác cao.
2. **Phân loại bệnh trên lá Táo:* So sánh hiệu quả giữa mô hình tự xây và các kiến trúc CNN danh tiếng có sẵn để rút ra bài học quan trọng về chiến lược xây dựng mô hình.

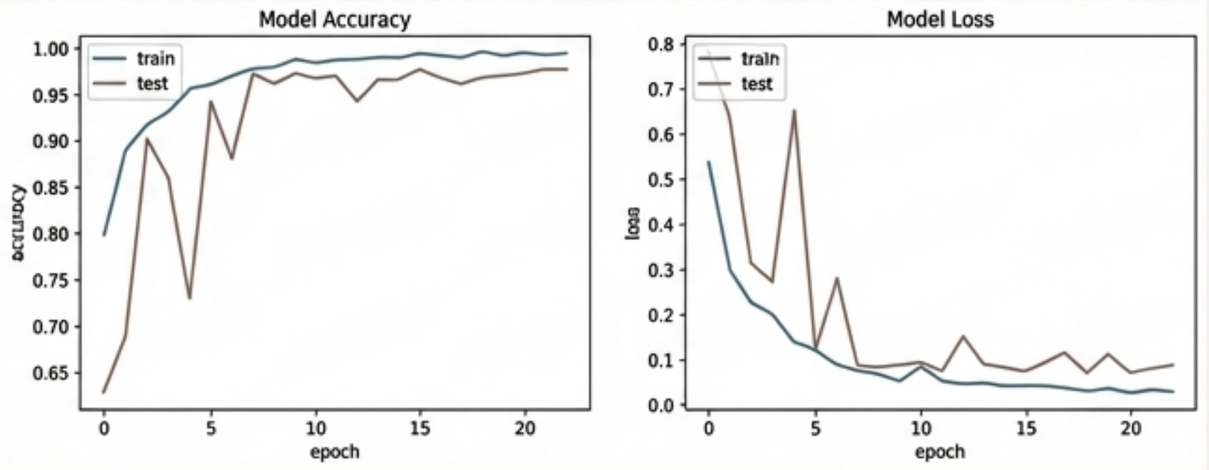
NotebookLM

Case Study 1: Bệnh trên lá Khoai tây (Dataset: Plant Village)

- **Bài toán:** Phân loại ảnh lá khoai tây thành 3 lớp:
  - Khỏe (Healthy)
  - Bệnh Cháy sớm (Early Blight)
  - Bệnh Cháy muộn (Late Blight)
- **Phương pháp:** Xây dựng một mô hình CNN tuần tự (Sequential) đơn giản bằng Keras.
- **Kiến trúc Mẫu (trích đoạn code):**

```
model = Sequential([
    # Đầu vào là ảnh 256x256x3
    BatchNormalization(input_shape=(256,256,3)),
    Convolution2D(32, (3,3), activation='relu'),
    MaxPooling2D((3,3)),
    Convolution2D(64, (3,3), activation='relu'),
    MaxPooling2D((3,3)),
    Flatten(),
    Dense(200, activation='relu'),
    Dense(3, activation='softmax') # 3 lớp đầu ra
])
```

Kết quả:



- **Thành tựu:** Độ chính xác trên tập kiểm tra (validation accuracy) đạt **97.68%**.
- **Nhận xét:** Một mô hình CNN đơn giản có thể đạt hiệu quả rất cao cho một bài toán được xác định rõ ràng.

NotebookLM

Case Study 2: Bệnh trên lá Táo & Bài học về 'Học chuyển giao'

- **Bài toán:** Phân loại 4 lớp bệnh trên lá táo, một bài toán phức tạp hơn (Dataset: Kashmir).
- **Phương pháp:** So sánh hiệu năng giữa mô hình tự xây (Custom CNN) và 3 kiến trúc nổi tiếng được huấn luyện trước (pre-trained): VGG16, ResNet-50, InceptionV3.

Bảng Kết quả So sánh:

Thuật toán	Độ chính xác (%)	Độ chuẩn xác (Precision)	Độ phủ (Recall)	Điểm F1
Custom CNN	80	0.78	0.77	0.77
VGG16	**85**	**0.82**	**0.81**	**0.81**
ResNet-50	81	0.79	0.77	0.78
InceptionV3	83	0.80	0.78	0.79

Bài học then chốt:

'Không cần phát minh lại bánh xe.'

Tận dụng các kiến trúc đã được chứng minh hiệu quả (kỹ thuật **Học chuyển giao - Transfer Learning**) thường mang lại kết quả tốt hơn và nhanh hơn so với việc tự xây dựng từ đầu, đặc biệt với dữ liệu phức tạp hoặc hạn chế.

NotebookLM

Hệ sinh thái Công cụ: Keras & TensorFlow

Các API, công cụ hỗ trợ và kỹ thuật tối ưu hóa để bạn bắt đầu hành trình của mình.

- Chúng ta sẽ khám phá ba khía cạnh chính của hệ sinh thái này:

* **3 API Xây dựng Mô hiih:** Lựa chọn công cụ phù hợp với độ phức tạp của bài toán.

* **Công cụ Hỗ trợ Huấn luyện:** Các ‘trợ lý’ thông minh giúp quá trình huấn luyện hiệu quả hơn.

* **Tối ưu hóa Tự động:** Các kỹ thuật để máy tính tự tìm ra cấu hình mô hình tốt nhất.

NotebookLM

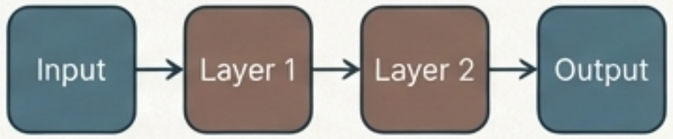
3 API Xây dựng Mô hình: Từ Đơn giản đến Tùy biến sâu

Sequential API

Mô tả: Xếp chồng các lớp theo một chuỗi duy nhất, như một chuỗi lego.

Khi nào dùng: Các mô hình CNN, MLP đơn giản. Dễ nhất để bắt đầu.

Độ khó: Dễ

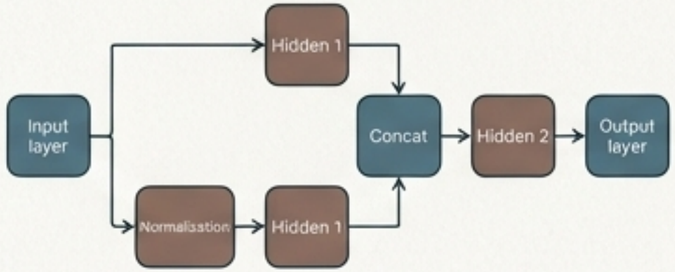


Functional API

Mô tả: Xây dựng các đồ thị lớp phức tạp, linh hoạt.

Khi nào dùng: Các kiến trúc có nhiều đầu vào/ra, các nhánh rẽ (ví dụ: Wide & Deep, ResNet).

Độ khó: Trung bình

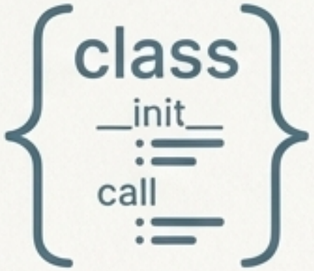


Subclassing API

Mô tả: Định nghĩa mô hình như một lớp (class) Python.

Khi nào dùng: Cần toàn quyền kiểm soát logic, phù hợp cho nghiên cứu và thử nghiệm các kiến trúc mới lạ.

Độ khó: Khó



NotebookLM

Huấn luyện Thông minh hơn: Callbacks & TensorBoard

Công cụ Hỗ trợ Quá trình Huấn luyện (Callbacks)

Callbacks là các 'trợ lý' tự động được gọi trong lúc `model.fit()`. Chúng giúp bạn giám sát và hành động dựa trên trạng thái của mô hình.

Ví dụ quan trọng:

- **`ModelCheckpoint`**: Tự động lưu lại mô hình tốt nhất (dựa trên `val_loss` hoặc `val_accuracy`). Không bao giờ làm mất mô hình tốt nhất của bạn.
- **`EarlyStopping`**: Dừng huấn luyện sớm khi mô hình không còn cải thiện, giúp tiết kiệm thời gian và chống overfitting hiệu quả.

Mục tiêu: Tự động hóa việc giám sát, giúp bạn tập trung vào thử nghiệm.



Trực quan hóa với TensorBoard

TensorBoard là một dashboard giúp bạn 'nhìn thấu' và phân tích quá trình học của mô hình một cách trực quan.

Tính năng chính:

📈 Theo dõi trực tiếp đường cong học tập (**Loss**, **Accuracy**) để xem mô hình có đang học tốt hay không.

🔬 So sánh hiệu quả giữa các lần chạy với siêu tham số khác nhau để tìm ra phương án tối ưu.

Mục tiêu: Hiểu sâu hơn về hành vi của mô hình, giúp gỡ lỗi và tối ưu hóa.

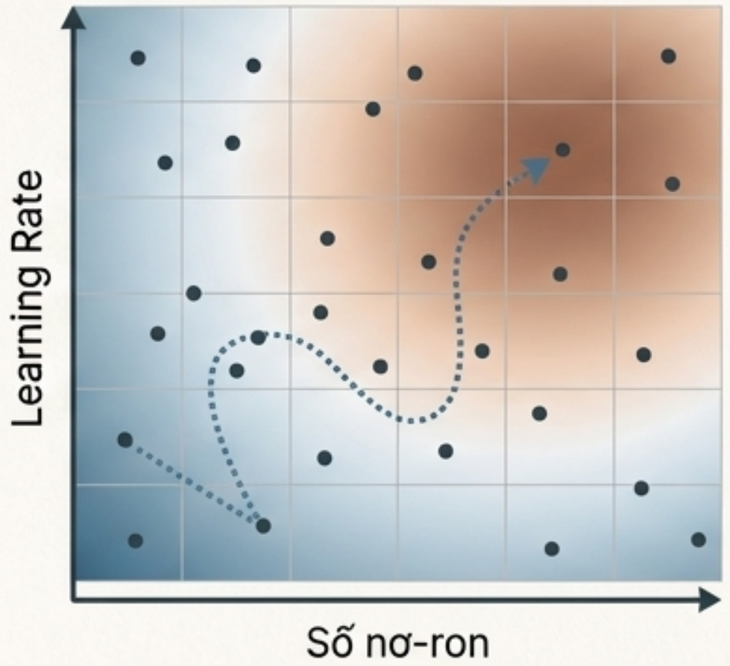


NotebookLM

Tối ưu hóa Siêu tham số: Từ 'Đoán mò' đến Tự động

Vấn đề: Làm sao để chọn được số lớp, số nơ-ron, tốc độ học (learning rate) tối ưu?

- Thử và sai thủ công: Tốn thời gian, không hiệu quả, dễ bỏ lỡ cấu hình tốt nhất. Quá trình này không có tính hệ thống và khó tái tạo.



Giải pháp: Keras Tuner: Một thư viện giúp tự động hóa quá trình tìm kiếm không gian siêu siêu tham số để tìm ra kiến trúc và cấu hình hiệu quả nhất cho bài toán của bạn.

Các chiến lược tìm kiếm phổ biến:

- **RandomSearch:** Thử ngẫu nhiên các kết hợp. Nhanh và đơn giản.
- **Hyperband:** Một thuật toán thông minh hơn, loại bỏ sớm các thử nghiệm kém hiệu quả để tập trung tài nguyên vào những “ứng viên” hứa hẹn nhất.
- **BayesianOptimization:** Dần dần học về các vùng siêu tham số tiềm năng để hội tụ nhanh hơn đến giải pháp tối ưu.

NotebookLM

Tóm tắt Phần 2 & Gợi ý Tiếp theo

Những gì chúng ta đã học:

1. **CNN là kiến trúc chủ lực** cho các bài toán thị giác máy tính nhờ khả năng học các đặc trưng không gian, vượt trội hơn ANN thông thường trong việc xử lý hình ảnh.

2. **Thực tiễn đã chứng minh** CNN có thể đạt độ chính xác rất cao trong việc phát hiện bệnh cây trồng. Việc sử dụng các kiến trúc có sẵn (Transfer Learning) là một chiến lược hiệu quả và tiết kiệm thời gian.

3. **Hệ sinh thái Keras/TensorFlow** cung cấp một bộ công cụ mạnh mẽ và toàn diện, từ việc xây dựng, huấn luyện, giám sát đến tối ưu hóa mô hình một cách chuyên nghiệp.

Hoạt động Gợi ý:

- **Live Demo:** Với nền tảng IT của lớp, chúng ta có thể xem một đoạn code Python mẫu (sử dụng Keras) để minh họa cho việc xây dựng mô hình CNN phát hiện bệnh trên lá khoai tây từ Case Study 1.
- **Hỏi & Đáp**

NotebookLM